David Goblot Prog P22

Document d'analyse

Algorithme génétique pour l'équilibrage d'un jeu carte type Magic

Le but de cet exercice est d'expérimenter avec des algorithmes génétiques pour donner aux Game Designers des outils d'équilibrage rationnel se basant sur des données. Dans notre cas, le jeu à équilibrer est un jeu de carte type *Magic* simplifié.

1. Description rapide du jeu et des cartes

Le jeu se joue en tour par tour, en un contre un. Chaque joueur.euse à 30 Points de Vie (PV), la partie est gagnée lorsque les PV adverses tombent à 0. Chaque joueur.euse possède un deck de 40 cartes sans doublon, et commence la partie en piochant 7 cartes, puis pioche une carte en début de tour mais perd un PV si son deck est vide. Les joueur.euses commencent avec 4 *Mana* à dépenser pendant leur tour, et regagne tout leur *Mana* en début de tour. Ielles augmentent également leur maximum de *Mana* de 1 à chaque tour.

Une carte à un coût en *Mana*, une valeur d'attaque, de défense, et jusqu'à trois capacités parmi **Garde, Vol** et **Charge.** Le coût d'un carte est défini par la formule :

FLOOR((Attaque + Défense) / 2 + Somme des coût de capacités)

Le prix maximum d'une carte est 7, avec une valeur d'attaque entre 0 et 14 et une valeur de défense entre 1 et 15. **Garde** coûte 2, **Vol** coûte 1 et **Charge** coûte 2. Cela donne une liste de cartes complète de **476 cartes uniques.**

Une carte doit attendre un tour après son invocation pour pouvoir attaquer, et une carte attaque directement les PV adverses. Une carte avec garde bloque les cartes adverses et entraîne un combat carte contre carte. Vol permet de passer au-dessus des cartes gardes et de toucher malgré tout les PV adverses (sauf si l'adversaire a une carte avec garde ET vol). Charge permet d'attaquer dès le tour d'invocation (donc ignorer le tour d'attente).

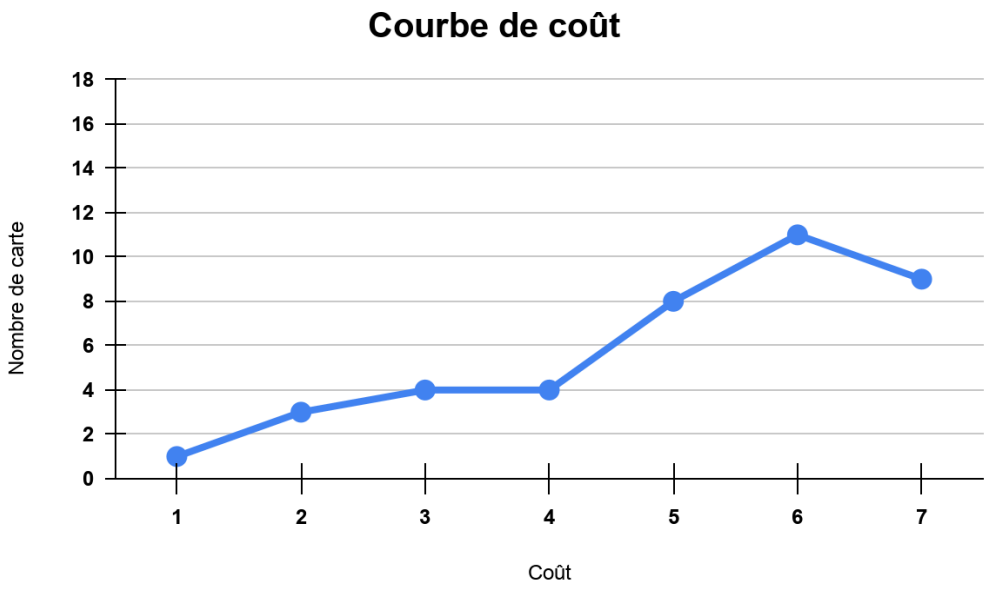
2. Description du protocole et de l'algorithme

L'objectif est de créer le meilleur deck possible afin d'avoir un aperçu de la "méta" avec les règles actuelles. Pour ça nous allons automatiser 500 combats entre J1 et J2, et optimiser le deck de J1.

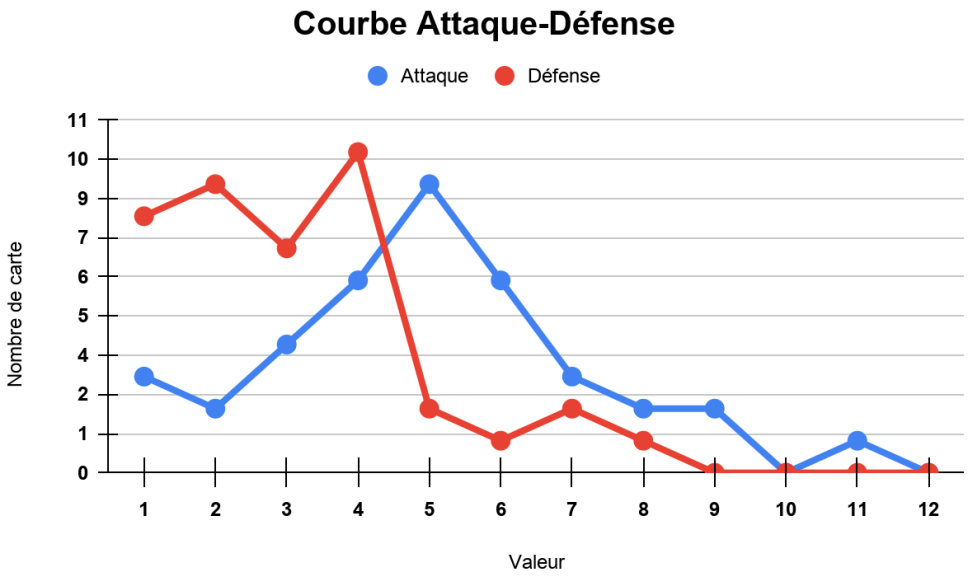
J1 et J2 commencent avec un deck aléatoire et font 500 combats, les 250 premiers avec J1 en tant que premier joueur, et on retient le *winrate* (nombre de victoire / 500) de J1. On recommence ce processus 10 000 fois mais entre chaque série, on va appliquer une **méthode de Monte-Carlo**. Si le *winrate* est inférieur au meilleur *winrate* obtenu jusqu'à présent, une carte est changée aléatoirement, on enlève une carte du deck et on ajoute une autre carte parmis toutes les cartes possibles pas encore présente dans le deck. Si le *winrate* est supérieur par contre, on le retient comme nouvel objectif. De plus, si le *winrate* dépasse 80%, on assigne le deck obtenu comme nouveau deck du J2 afin d'éviter la stagnation de l'algorithme.

3. Analyse des résultats

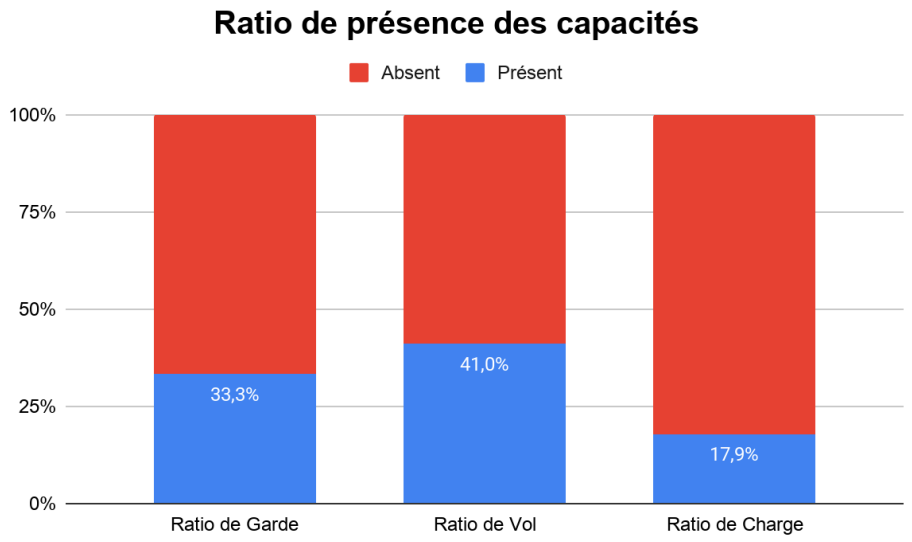
Les résultats sont montrés sous formes de plusieurs graphiques représentant la composition du deck final après les 10 000 itérations. (Lien vers le tableur)



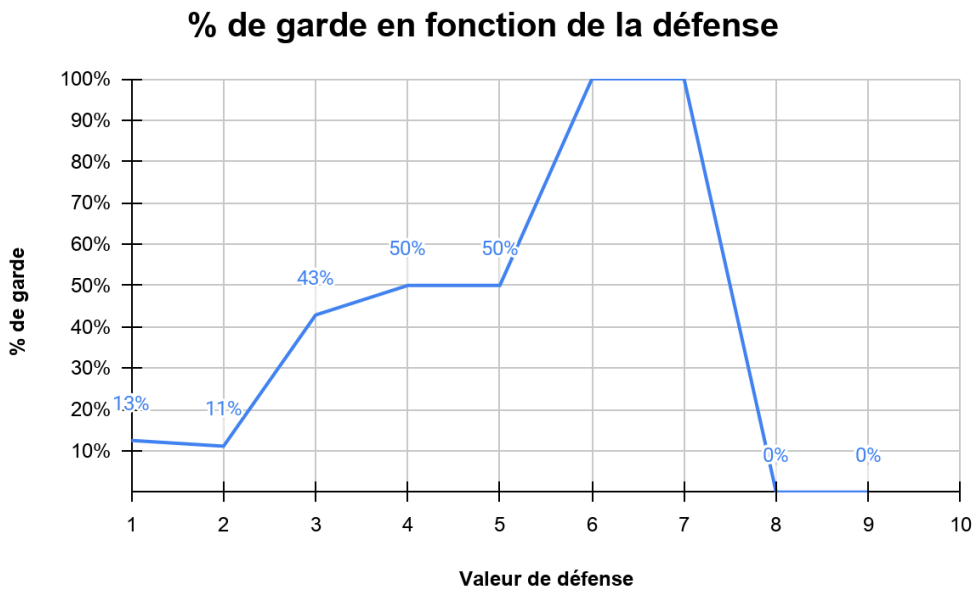
La courbe de coût montre que les cartes à un coût élevé sont plutôt priorisées, avec un pic à 6 mana, sûrement car les joueurs peuvent assez rapidement jouer les cartes les plus fortes (coût de 7) en commençant avec 4 mana. Quelques cartes plus faibles sont tout de même conservées afin d'avoir de grandes chances de jouer une carte dès le premier tour.



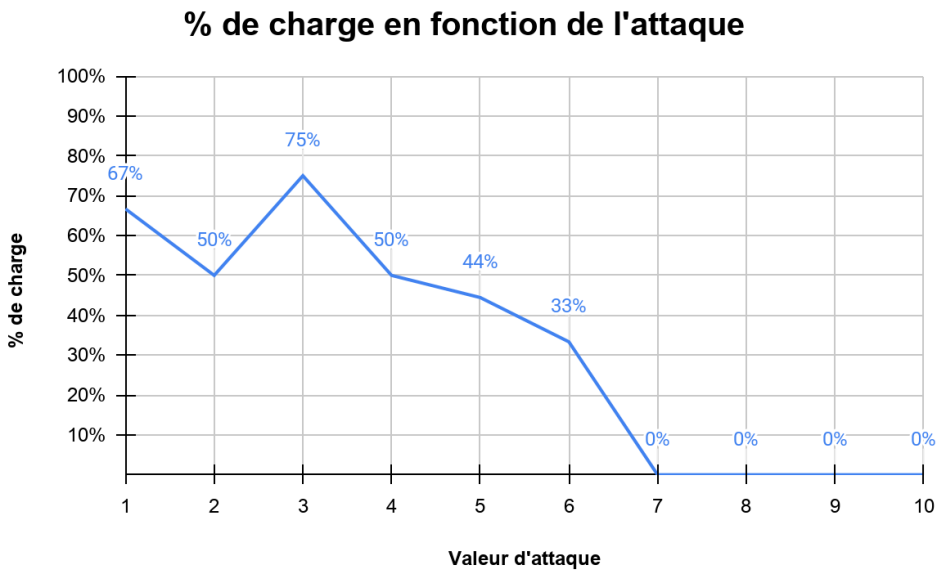
La courbe de la répartition des valeurs d'attaque et de défense du deck indique bien que la défense est une moins grande priorité que l'attaque, vu leur impact respectif sur le coût de la carte. Cependant le pic de l'attaque est à 5, sûrement pour compenser le prix des compétences ajoutées.



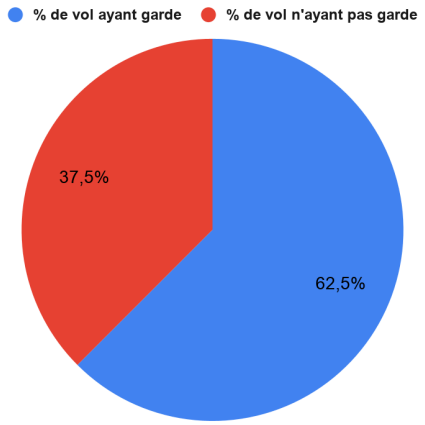
Le graphique des compétences présentes dans le deck indique que la compétence **Charge**, seulement 17,9% de présence, n'augmente pas beaucoup la qualité de la carte, ou que son coût est trop élevé. Les compétences **Garde** et **Vol** par contre sont assez bien mises en valeur, avec un ratio de présence de 33,3% et 41,0% respectivement.

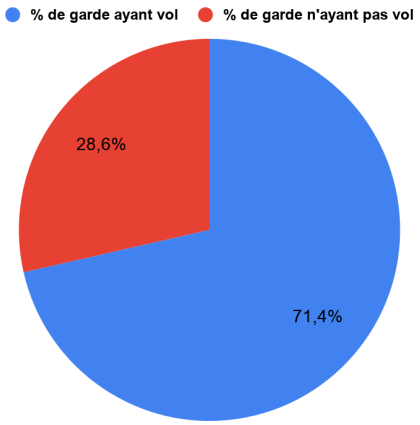


Ce graphique du pourcentage de **Garde** en fonction de la valeur de défense montre que la compétence garde est majoritairement utilisée sur les cartes ayant une forte valeur de défense, donc pouvant encaisser des attaques plus fortes et nombreuses. L'absence de cartes avec une très forte défense peut venir du prix trop élevé des cartes.



Avec ce graphique du pourcentage de **Charge** en fonction de la valeur d'attaque, on voit que cette compétence est mise en avant et brille sur des cartes plus faibles. Ces cartes peuvent être invoquées plus tôt dans la partie et tuer l'adversaire avant qu'il n'est le temps de réagir, et la compétence charge améliore cette stratégie.





Ces deux derniers graphiques analysent la co-occurrence entre les capacités **Garde** et **Vol**, car elles ont un comportement spécifique lorsqu'elles sont mises en commun sur une même carte (la possibilité de garder les attaques volantes). On voit donc que les cartes avec la compétence garde uniquement ne sont pas très favorisées, par rapport aux cartes avec la garde volante. Les cartes avec la compétence vol par contre ont un peu plus de valeur en tant que tel, sûrement dû au faible coût de cette compétence (1 mana).

4. Analyse critique

Pour cet exercice, j'ai uniquement utilisé la méthode de Monte-Carlo basique, avec une mutation aléatoire, mais plusieurs amélioration sont possible :

- Ajout d'un score dans la liste de carte, pour prendre en priorité les cartes qui améliore le winrate
- Implémentation d'algorithme génétique différent, comme un cross-over avec une évolution de plusieurs deck en parallèle

La limite principale de la méthode Monte-Carlo primitive est la tendance à ne pas trouver les stratégies impliquant plusieurs mutations individuellement faibles, mais fortes lorsqu'elles sont combinées. Cela ne devrait pas s'appliquer à notre jeu de carte qui reste très basique sans système de combo, mais cela aurait pu entraîner un choix plus appuyé de cartes avec une très forte attaque et peu de défense, et vice versa. L'avantage indéniable est la rapidité d'exécution sur plusieurs milliers d'itération, ce qui peut quasiment garantir de trouver les cartes fortes individuellement. Mon algorithme, par exemple, exécute les 10 000 itérations en 1min53.